Unit 4:
Introduction to Arrays, Structures, Pointers, Files, Functions, Recursion

Array and Strings (already shared a separate document)
Functions and Recursion (This Document)
Pointers (Separate Document)

User Defined Functions in C

A function is a block of code that performs a specific task.

C allows you to define functions according to your need. These functions are known as user-defined functions.
For example: Suppose, you need to create a circle and color it depending upon the radius and color. You can create two functions to solve this problem:

createCircle() function
color() function

Example: User-defined function

Here is an example to add two integers. To perform this task, we have created an user-defined addNumbers().

```
#include <stdio.h>
int addNumbers(int a, int b);          // function prototype

int main()
{
    int n1,n2,sum;

    printf("Enters two numbers: ");
    scanf("%d %d",&n1,&n2);

    sum = addNumbers(n1, n2);         // function call
    printf("sum = %d",sum);

    return 0;
}

int addNumbers(int a, int b)          // function definition
{
    int result;
    result = a+b;
```

```
    return result;                  // return statement
}
```

Function prototype

A function prototype is simply the declaration of a function that specifies the function's name, parameters and return type. It doesn't contain the function body.

A function prototype gives information to the compiler that the function may later be used in the program.

Syntax of function prototype:

```
returnType functionName(type1 argument1, type2 argument2, ...);
```

In the above example, int addNumbers(int a, int b); is the function prototype which provides the following information to the compiler:

1. name of the function is addNumbers()
2. return type of the function is int
3. two arguments of type int are passed to the function

Note:The function prototype is not needed if the user-defined function is defined before the main() function.

Calling a function

Control of the program is transferred to the user-defined function by calling it.

Syntax of function call

```
functionName(argument1, argument2, ...);
```

In the above example, the function call is made using the addNumbers(n1, n2); statement inside the main() function.

Function definition

Function definition contains the block of code to perform a specific task. In our example, adding two numbers and returning it.

Syntax of function definition

```
returnType functionName(type1 argument1, type2 argument2, ...)
{
```

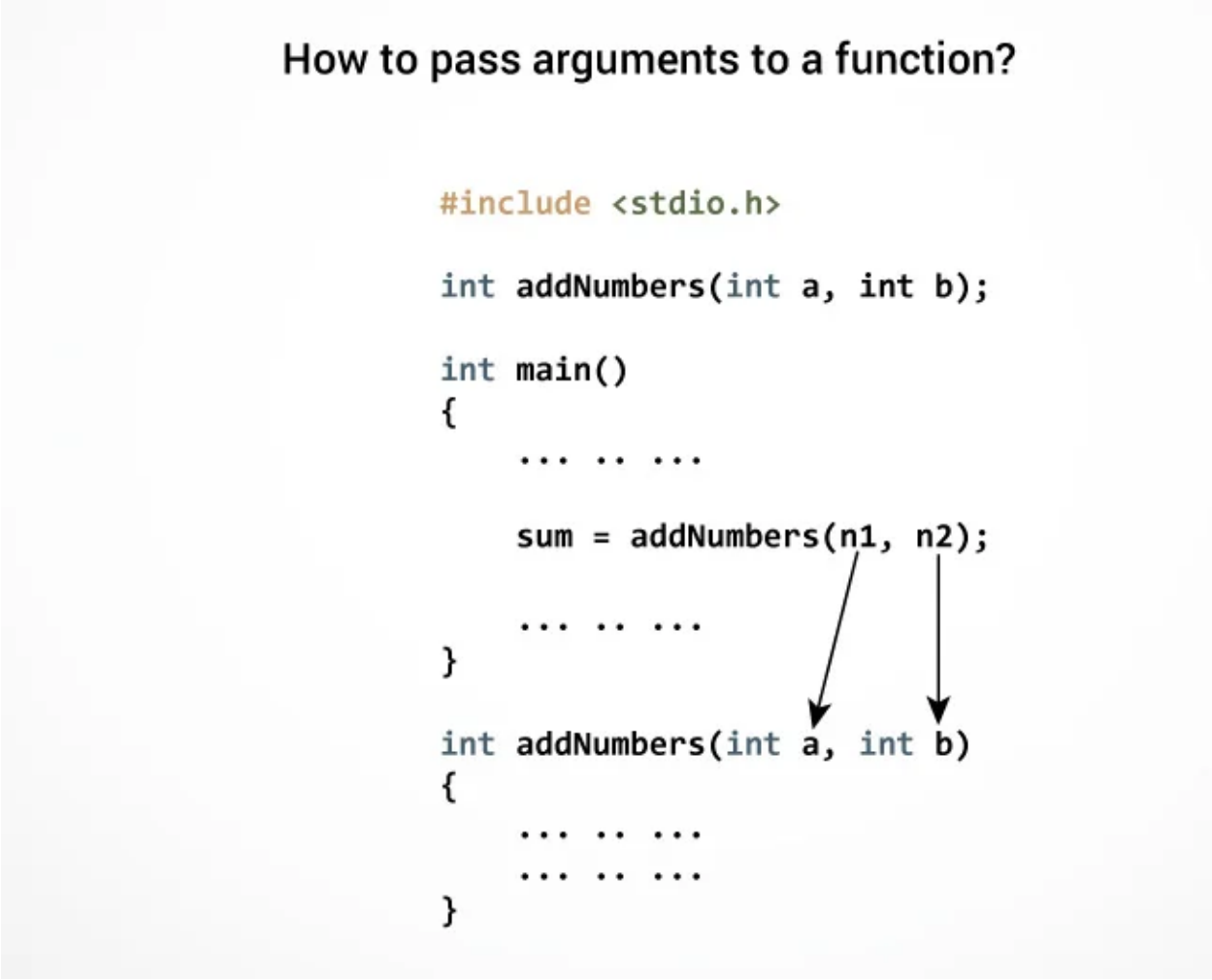
```
    //body of the function
}
```

When a function is called, the control of the program is transferred to the function definition. And, the compiler starts executing the codes inside the body of a function.

Passing arguments to a function

In programming, argument refers to the variable passed to the function. In the above example, two variables n1 and n2 are passed during the function call.

The parameters a and b accepts the passed arguments in the function definition. These arguments are called formal parameters of the function.



Passing arguments to a function

Passing Argument to Function

The type of (and number of) arguments passed to a function and the formal parameters must match, otherwise, the compiler will throw an error.

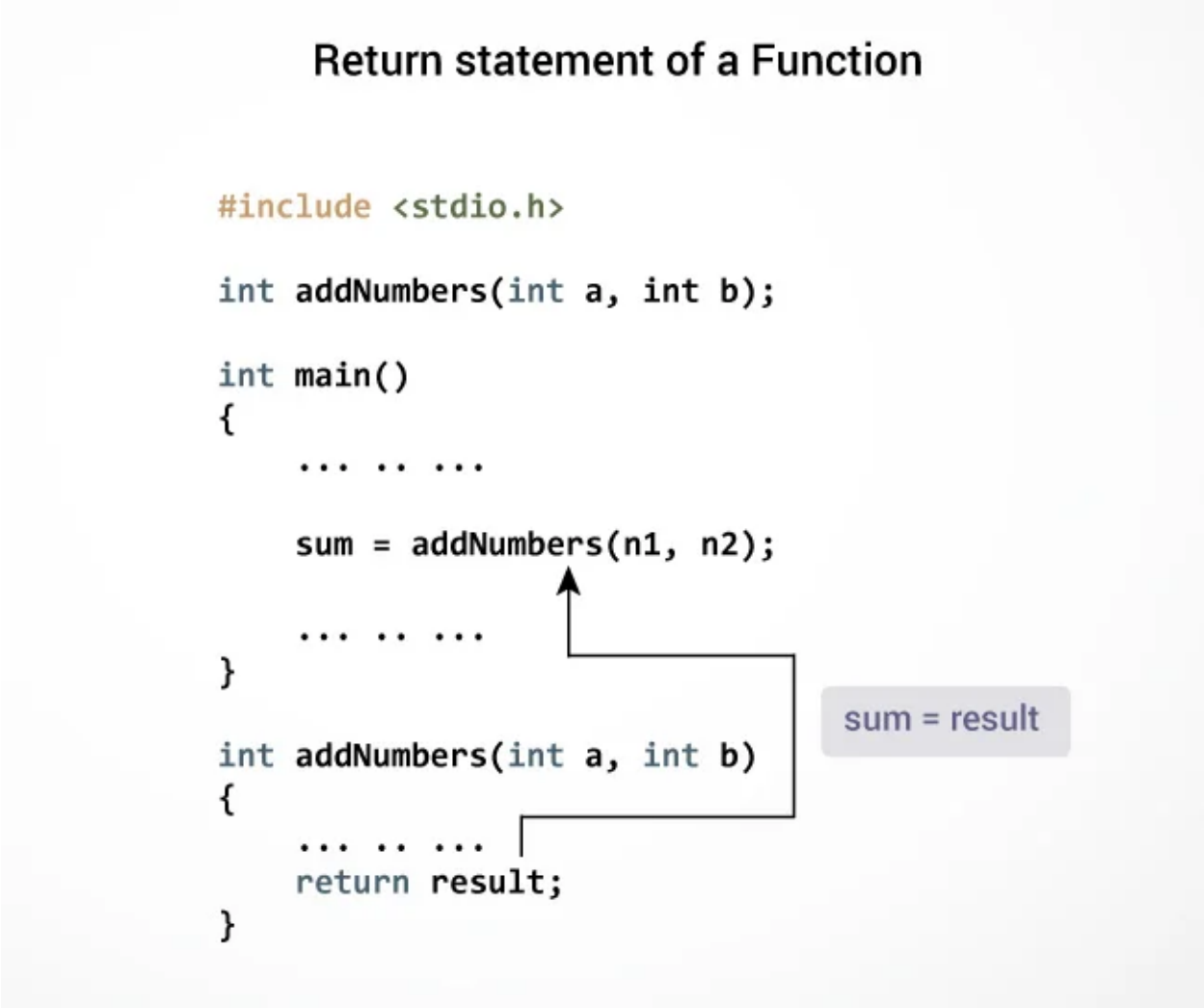
If n1 is of char type, a also should be of char type. If n2 is of float type, variable b also should be of float type.

A function can also be called without passing an argument.

Return Statement

The return statement terminates the execution of a function and returns a value to the calling function. The program control is transferred to the calling function after the return statement.

In the above example, the value of the result variable is returned to the main function. The `sum` variable in the `main()` function is assigned this value.



Return statement of a function

Syntax of return statement

```
return (expression);
```

For example,

```
return a;
return (a+b);
```

The type of value returned from the function and the return type specified in the function prototype and function definition must match.

Types of User-defined Functions in C Programming

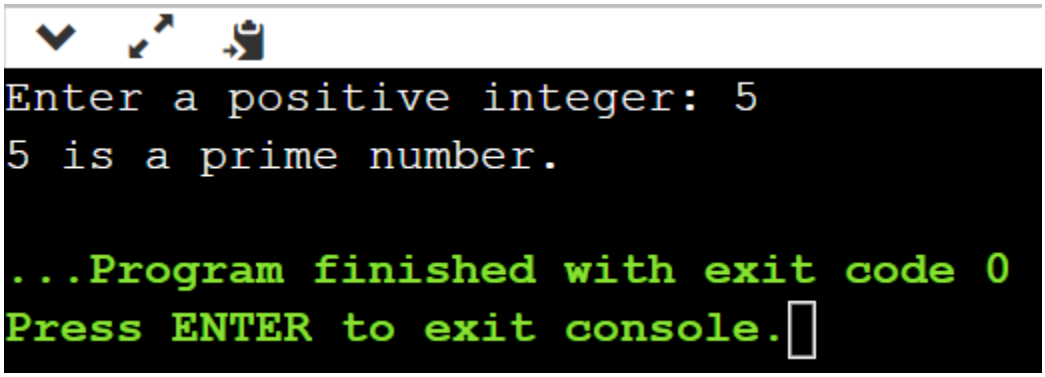
These 4 programs below check whether the integer entered by the user is a prime number or not.

The output of all these programs below is the same, and we have created a user-defined function in each example. However, the approach we have taken in each example is different.

Example 1: No Argument Passed and No Return Value

```
#include <stdio.h>

void checkPrimeNumber();

int main() {
  checkPrimeNumber();    // argument is not passed
  return 0;
}

// return type is void meaning doesn't return any value
void checkPrimeNumber() {
  int n, i, flag = 0;

  printf("Enter a positive integer: ");
  scanf("%d",&n);

  // 0 and 1 are not prime numbers
  if (n == 0 || n == 1)
    flag = 1;

  for(i = 2; i <= n/2; ++i) {
    if(n%i == 0) {
      flag = 1;
```

```
      break;
    }
  }

  if (flag == 1)
    printf("%d is not a prime number.", n);
  else
    printf("%d is a prime number.", n);
}
```



The checkPrimeNumber() function takes input from the user, checks whether it is a prime number or not, and displays it on the screen.

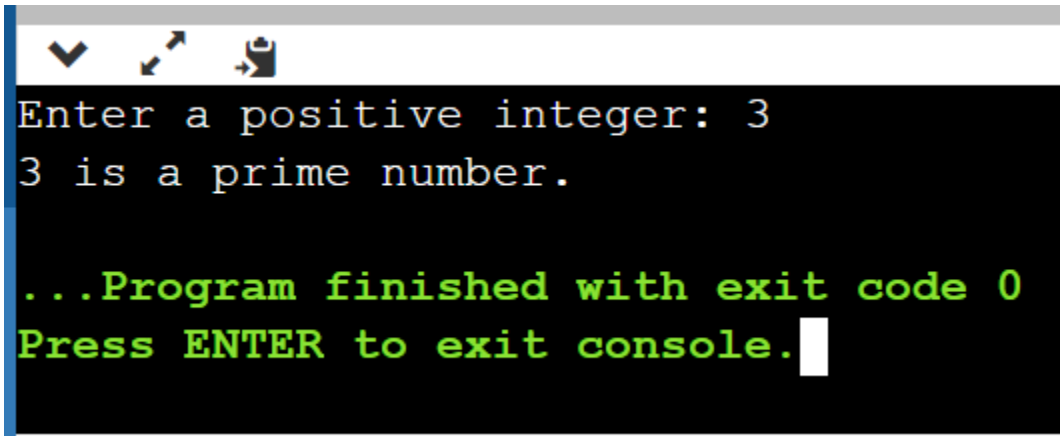
The empty parentheses in checkPrimeNumber(); inside the main() function indicates that no argument is passed to the function.

The return type of the function is void. Hence, no value is returned from the function.

Example 2: No Arguments Passed But Returns a Value

```
#include <stdio.h>
int getInteger();

int main() {

  int n, i, flag = 0;

  // no argument is passed
  n = getInteger();

  // 0 and 1 are not prime numbers
  if (n == 0 || n == 1)
    flag = 1;
```

```
  for(i = 2; i <= n/2; ++i) {
    if(n%i == 0){
      flag = 1;
      break;
    }
  }

  if (flag == 1)
    printf("%d is not a prime number.", n);
  else
    printf("%d is a prime number.", n);

  return 0;
}

// returns integer entered by the user
int getInteger() {
  int n;

  printf("Enter a positive integer: ");
  scanf("%d",&n);

  return n;
}
```

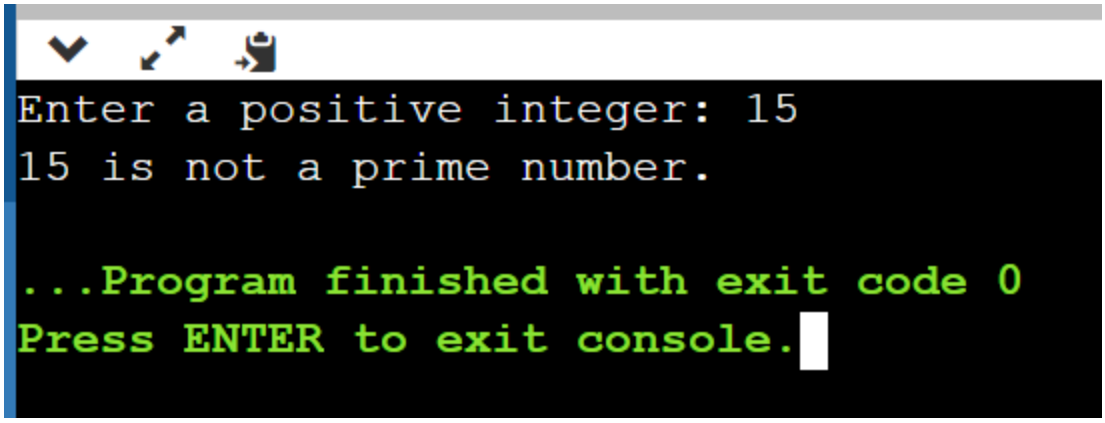


The empty parentheses in the n = getInteger(); statement indicates that no argument is passed to the function. And, the value returned from the function is assigned to n.

Here, the getInteger() function takes input from the user and returns it. The code to check whether a number is prime or not is inside the main() function.

Example 3: Argument Passed But No Return Value

```c
#include <stdio.h>
void checkPrimeAndDisplay(int n);

int main() {

  int n;

  printf("Enter a positive integer: ");
  scanf("%d",&n);

  // n is passed to the function
  checkPrimeAndDisplay(n);

  return 0;
}

// return type is void meaning doesn't return any value
void checkPrimeAndDisplay(int n) {
  int i, flag = 0;

  // 0 and 1 are not prime numbers
  if (n == 0 || n == 1)
    flag = 1;

  for(i = 2; i <= n/2; ++i) {
    if(n%i == 0){
      flag = 1;
      break;
    }
  }

  if(flag == 1)
    printf("%d is not a prime number.",n);
  else
    printf("%d is a prime number.", n);
}
```



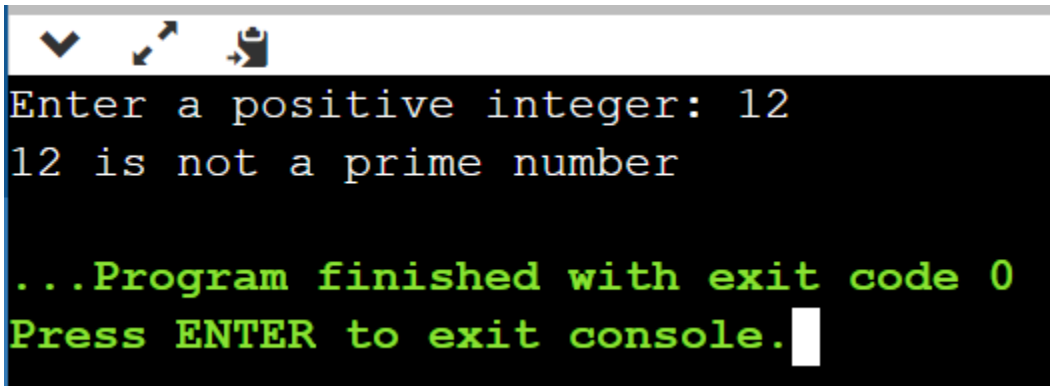
The integer value entered by the user is passed to the `checkPrimeAndDisplay()` function.

Here, the `checkPrimeAndDisplay()` function checks whether the argument passed is a prime number or not and displays the appropriate message.

Example 4: Argument Passed and Returns a Value

```
#include <stdio.h>
int checkPrimeNumber(int n);

int main() {

  int n, flag;

  printf("Enter a positive integer: ");
  scanf("%d",&n);

  // n is passed to the checkPrimeNumber() function
  // the returned value is assigned to the flag variable
  flag = checkPrimeNumber(n);

  if(flag == 1)
    printf("%d is not a prime number",n);
  else
    printf("%d is a prime number",n);

  return 0;
}

// int is returned from the function
int checkPrimeNumber(int n) {
```

```
  // 0 and 1 are not prime numbers
  if (n == 0 || n == 1)
    return 1;

  int i;

  for(i=2; i <= n/2; ++i) {
    if(n%i == 0)
      return 1;
  }

  return 0;
}
```



The input from the user is passed to the `checkPrimeNumber()` function.

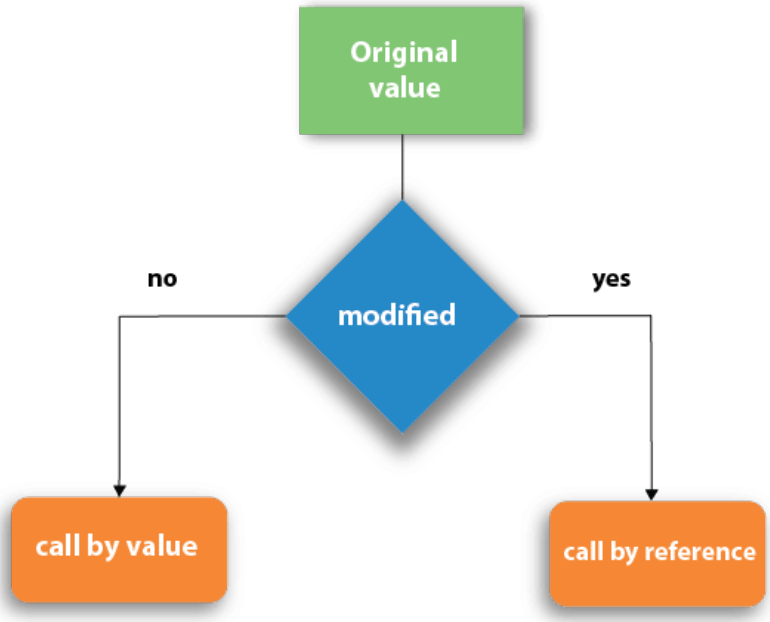
The `checkPrimeNumber()` function checks whether the passed argument is prime or not.

If the passed argument is a prime number, the function returns 0. If the passed argument is a non-prime number, the function returns 1. The return value is assigned to the `flag` variable.

Depending on whether the flag is 0 or 1, an appropriate message is printed from the `main()` function.

Argument passing in function calls: Call by reference and Call by Value

There are two methods to pass the data into the function in C language, i.e., call by value and call by reference.



Call by value in C

- In the call by value method, the value of the actual parameters is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.
- In the call by value method, we can not modify the value of the actual parameter by the formal parameter.
- In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.
- The actual parameter is the argument which is used in the function call whereas the formal parameter is the argument which is used in the function definition.

Let's try to understand the concept of call by value in c language by the example given below:

```
#include<stdio.h>
void change(int num) {
    printf("Before adding value inside function num=%d \n",num);
    num=num+100;
    printf("After adding value inside function num=%d \n", num);
}
int main() {
    int x=100;
    printf("Before function call x=%d \n", x);
    change(x);//passing value in function
    printf("After function call x=%d \n", x);
return 0;
}
```

```c
#include<stdio.h>
void change(int num) {
    printf("Before adding value inside function num=%d \n",num);
    num=num+100;
    printf("After adding value inside function num=%d \n", num);
}
int main() {
    int x=100;
    printf("Before function call x=%d \n", x);
    change(x);//passing value in function
    printf("After function call x=%d \n", x);
return 0;
}
```

input

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=100


...Program finished with exit code 0
Press ENTER to exit console.
```

Another Call by Value Example: Swapping the values of the two variables

```c
#include <stdio.h>
void swap(int , int); //prototype of the function
int main()
{
    int a = 10;
    int b = 20;
    printf("Before swapping the values in main a = %d, b = %d\n",a,b);
// printing the value of a and b in main
    swap(a,b);
    printf("After swapping values in main a = %d, b = %d\n",a,b); //
The value of actual parameters do not change by changing the formal
parameters in call by value, a = 10, b = 20
}
void swap (int a, int b)
{
    int temp;
```

```
    temp = a;
    a=b;
    b=temp;
    printf("After swapping values in function a = %d, b = %d\n",a,b);
// Formal parameters, a = 20, b = 10
}
```

```c
#include <stdio.h>
void swap(int , int); //prototype of the function
int main()
{
    int a = 10;
    int b = 20;
    printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the value of a and b in main
    swap(a,b);
    printf("After swapping values in main a = %d, b = %d\n",a,b);
    //The value of actual parameters do not change by
    //changing the formal parameters in call by value, a = 10, b = 20
}
void swap (int a, int b)
{
    int temp;
    temp = a;
    a=b;
    b=temp;
    printf("After swapping values in function a = %d, b = %d\n",a,b); // Formal parameters, a = 20, b = 10
}
```

```
Before swapping the values in main a = 10, b = 20
After swapping values in function a = 20, b = 10
After swapping values in main a = 10, b = 20


...Program finished with exit code 0
Press ENTER to exit console.
```

Call by reference in C

- In call by reference, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.

Consider the following example for the call by reference.

```
#include<stdio.h>
```

```
void change(int *num) {
    printf("Before adding value inside function num=%d \n",*num);
    (*num) += 100;
    printf("After adding value inside function num=%d \n", *num);
}
int main() {
    int x=100;
    printf("Before function call x=%d \n", x);
    change(&x);//passing reference in function
    printf("After function call x=%d \n", x);
return 0;
}
```

```
#include<stdio.h>
void change(int *num) {
    printf("Before adding value inside function num=%d \n",*num);
    (*num) += 100;
    printf("After adding value inside function num=%d \n", *num);
}
int main() {
    int x=100;
    printf("Before function call x=%d \n", x);
    change(&x);//passing reference in function
    printf("After function call x=%d \n", x);
return 0;
}
```

input

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=200


...Program finished with exit code 0
Press ENTER to exit console.
```

Another Call by reference Example: Swapping the values of the two variables

```
#include <stdio.h>
void swap(int *, int *); //prototype of the function
int main()
{
    int a = 10;
    int b = 20;
```

```
    printf("Before swapping the values in main a = %d, b = %d\n",a,b);
// printing the value of a and b in main
    swap(&a,&b);
    printf("After swapping values in main a = %d, b = %d\n",a,b); //
The values of actual parameters do change in call by reference, a =
10, b = 20
}
void swap (int *a, int *b)
{
    int temp;
    temp = *a;
    *a=*b;
    *b=temp;
    printf("After swapping values in function a = %d, b =
%d\n",*a,*b); // Formal parameters, a = 20, b = 10
}
```

```c
#include <stdio.h>
void swap(int *, int *); //prototype of the function
int main()
{
    int a = 10;
    int b = 20;
    printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the value of a and b in main
    swap(&a,&b);
    printf("After swapping values in main a = %d, b = %d\n",a,b);
    // The values of actual parameters do change in call by reference, a = 10, b = 20
}
void swap (int *a, int *b)
{
    int temp;
    temp = *a;
    *a=*b;
    *b=temp;
    printf("After swapping values in function a = %d, b = %d\n",*a,*b); // Formal parameters, a = 20, b = 10
}
```

input

```
Before swapping the values in main a = 10, b = 20
After swapping values in function a = 20, b = 10
After swapping values in main a = 20, b = 10


...Program finished with exit code 0
Press ENTER to exit console.
```

Difference between call by value and call by reference in c

No.	Call by value	Call by reference
1	A copy of the value is passed into the function	An address of value is passed into the function
2	Changes made inside the function are limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.
3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location

Recursion in C

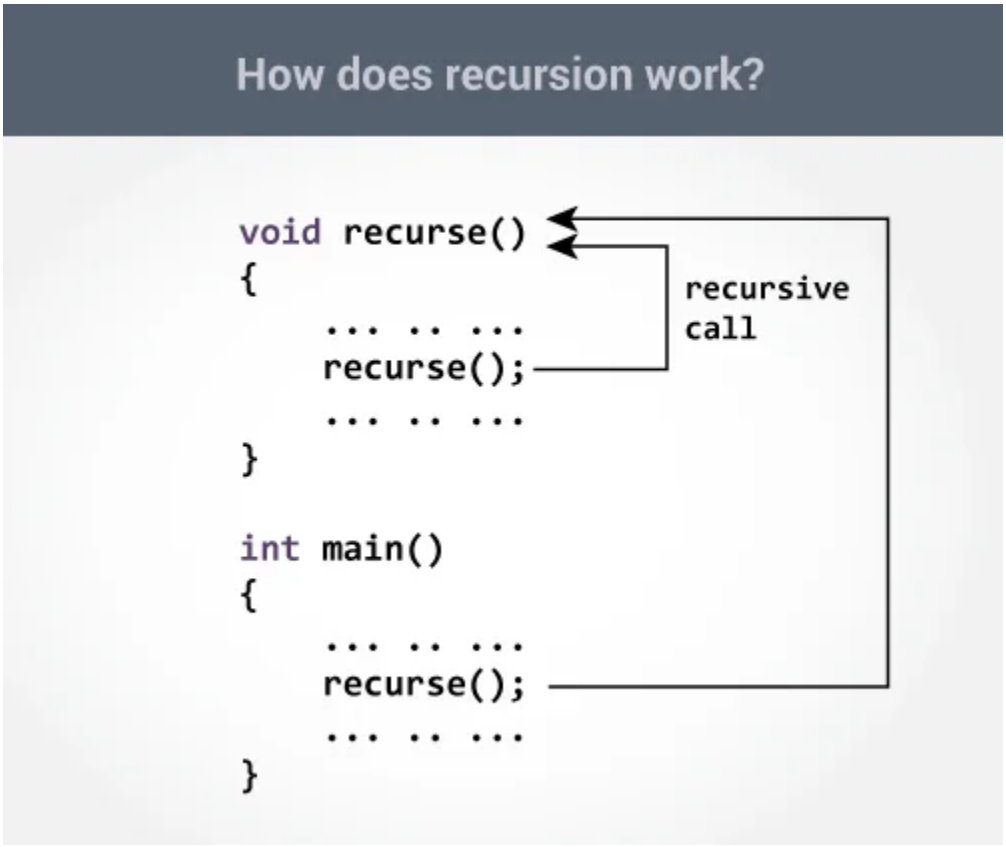
Recursion is the process which comes into existence when a function calls a copy of itself to work on a smaller problem. Any function which calls itself is called a recursive function, and such function calls are called recursive calls. Recursion involves several numbers of recursive calls. However, it is important to impose a termination condition of recursion. Recursion code is shorter than iterative code however it is difficult to understand.

Recursion cannot be applied to all the problems, but it is more useful for the tasks that can be defined in terms of similar subtasks. For Example, recursion may be applied to sorting, searching, and traversal problems.

Generally, iterative solutions are more efficient than recursion since function calls are always overhead. Any problem that can be solved recursively, can also be solved iteratively. However, some problems are best suited to be solved by the recursion, for example, tower of Hanoi, Fibonacci series, factorial finding, etc.

How does recursion work?

```
void recurse()
{
    ... .. ...
    recurse();
    ... .. ...
}

int main()
{
    ... .. ...
    recurse();
    ... .. ...
}
```



The recursion continues until some condition is met to prevent it.

To prevent infinite recursion, if...else statement (or similar approach) can be used where one branch makes the recursive call, and another doesn't.

Example: Sum of Natural Numbers Using Recursion

```
#include <stdio.h>
```

```c
int sum(int n);

int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);

    result = sum(number);

    printf("sum = %d", result);
    return 0;
}

int sum(int n) {
    if (n != 0)
        // sum() function calls itself
        return n + sum(n-1);
    else
        return n;
}
```

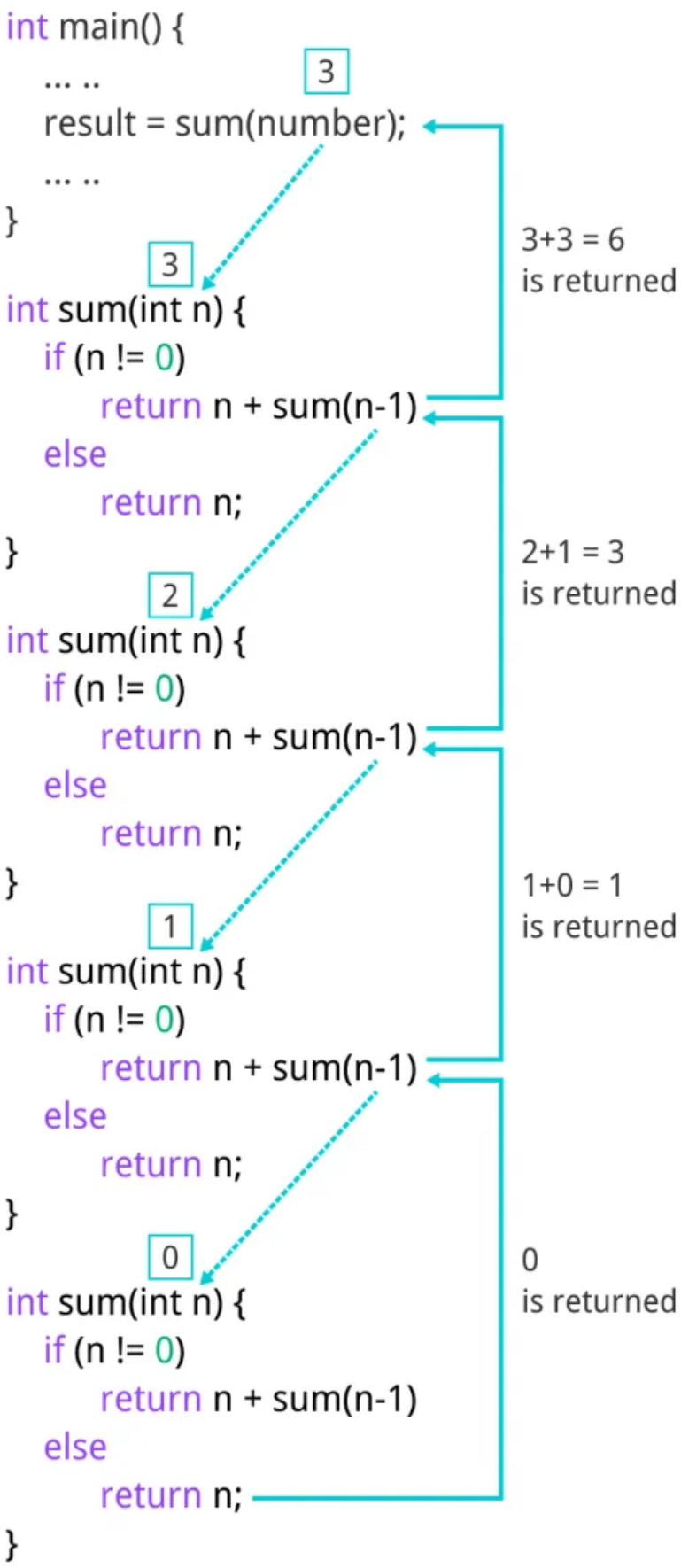
```c
#include <stdio.h>
int sum(int n);

int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);

    result = sum(number);

    printf("sum = %d", result);
    return 0;
}

int sum(int n) {
    if (n != 0)
        // sum() function calls itself
        return n + sum(n-1);
    else
        return n;
}
```

```
Enter a positive integer: 5
sum = 15

...Program finished with exit code 0
Press ENTER to exit console.
```

Initially, the `sum()` is called from the `main()` function with the number passed as an argument.

Suppose, the value of n inside `sum()` is 3 initially. During the next function call, 2 is passed to the sum() function. This process continues until n is equal to 0.

When n is equal to 0, the if condition fails and the else part is executed returning the sum of integers ultimately to the main() function.


int main() {
... ..
result = sum(number);
... ..
}
3
int sum(int n) {
if (n != 0)
return n + sum(n-1)
else
return n;
}
3
3+3 = 6
is returned
int sum(int n) {
if (n != 0)
return n + sum(n-1)
else
return n;
}
2
2+1 = 3
is returned
int sum(int n) {
if (n != 0)
return n + sum(n-1)
else
return n;
}
1
1+0 = 1
is returned
int sum(int n) {
if (n != 0)
return n + sum(n-1)
else
return n;
}
0
0
is returned
int sum(int n) {
if (n != 0)
return n + sum(n-1)
else
return n;
}

In the following example, recursion is used to calculate the factorial of a number.

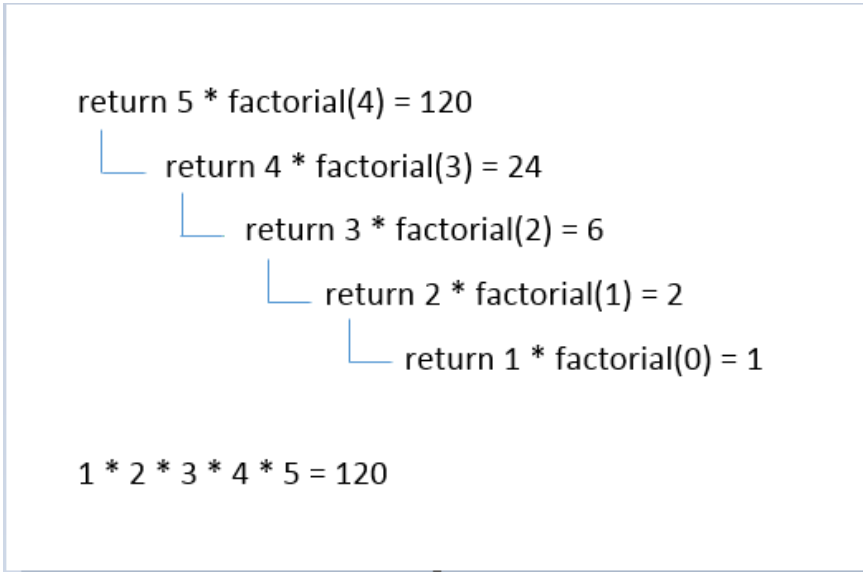
```
#include <stdio.h>
int fact (int);
int main()
{
    int n,f;
    printf("Enter the number whose factorial you want to calculate?");
    scanf("%d",&n);
    f = fact(n);
    printf("factorial = %d",f);
}
int fact(int n)
{
    if (n==0)
    {
        return 0;
    }
    else if ( n == 1)
    {
        return 1;
    }
    else
    {
        return n*fact(n-1);
    }
}
```

```c
#include <stdio.h>
int fact (int);
int main()
{
    int n,f;
    printf("Enter the number whose factorial you want to calculate?");
    scanf("%d",&n);
    f = fact(n);
    printf("factorial = %d",f);
}
int fact(int n)
{
    if (n==0)
    {
        return 0;
    }
    else if ( n == 1)
    {
        return 1;
    }
    else
    {
        return n*fact(n-1);
    }
}
```

input

```
Enter the number whose factorial you want to calculate?5
factorial = 120
```

We can understand the above program of the recursive method call by the figure given below:



Recursive Function

A recursive function performs the tasks by dividing it into the subtasks. There is a termination condition defined in the function which is satisfied by some specific subtask. After this, the recursion stops and the final result is returned from the function.

The case at which the function doesn't recur is called the base case whereas the instances where the function keeps calling itself to perform a subtask, is called the recursive case. All the recursive functions can be written using this format.

Pseudocode for writing any recursive function is given below.

```
if (test_for_base)
{
    return some_value;
}
else if (test_for_another_base)
{
    return some_another_value;
}
else
{
    // Statements;
    recursive call;
}
```

Another Example of recursion in C

Let's see an example to find the nth term of the Fibonacci series.

```
#include<stdio.h>
int fibonacci(int);
void main ()
{
    int n,f;
    printf("Enter the value of n?");
    scanf("%d",&n);
    f = fibonacci(n);
    printf("%d",f);
}
int fibonacci (int n)
{
    if (n==0)
    {
    return 0;
    }
    else if (n == 1)
    {
```

```
        return 1;
    }
    else
    {
        return fibonacci(n-1)+fibonacci(n-2);
    }
}
```

```
#include<stdio.h>
int fibonacci(int);
void main ()
{
    int n,f;
    printf("Enter the value of n?");
    scanf("%d",&n);
    f = fibonacci(n);
    printf("%d",f);
}
int fibonacci (int n)
{
    if (n==0)
    {
    return 0;
    }
    else if (n == 1)
    {
        return 1;
    }
    else
    {
        return fibonacci(n-1)+fibonacci(n-2);
    }
}
```

```
Enter the value of n?12
144
```